

**MALABR (MACHINE LeARNING in BrowSeR) - Leveraging
Native System Resources for Machine Learning Capabilities
within Web Browser**

*Thesis to be submitted in partial fulfillment of the
requirements for the degree*

of

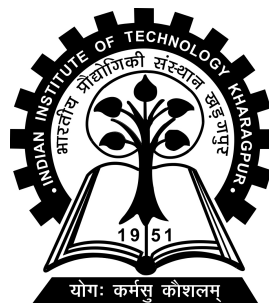
Master of Technology

by

**Saurav
23CS60R32**

Under the guidance of

Dr Mainack Mondal



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR**



Department of Computer Science
and Engineering
Indian Institute of Technology,
Kharagpur
India - 721302

CERTIFICATE

This is to certify that we have examined the thesis entitled **MALABR (MACHine LeArning in BrowseR) - Leveraging Native System Resources for Machine Learning Capabilities within Web Browser**, submitted by **Saurav** (Roll Number: *23CS60R32*), a postgraduate student of the **Department of Computer Science and Engineering**, in partial fulfillment for the award of the degree of Master of Technology. We hereby accord our approval of it as a study carried out and presented in a manner required for its acceptance in partial fulfillment for the Postgraduate Degree for which it has been submitted. The thesis has fulfilled all the requirements as per the regulations of the Institute and has reached the standard needed for submission.

Dr. Mainack Mondal

Department of Computer
Science and Engineering
Indian Institute of Technology,
Kharagpur

Place: Kharagpur

Date:

ACKNOWLEDGEMENTS

I would like to express my deepest gratitude to my supervisor, Dr. Mainack Mondal, for his invaluable guidance, encouragement, and insightful feedback throughout this project. His expertise and support have been instrumental in shaping the direction and scope of this work, and his mentorship has been an invaluable resource in overcoming the challenges encountered during this research.

I am also sincerely grateful to Prof. Ayan Chaudhury, my faculty advisor, for his continuous support and advice. His suggestions and feedback have greatly contributed to refining and improving this project, and his guidance has been a steady source of motivation.

I would like to thank the Department of Computer Science and Engineering at the Indian Institute of Technology, Kharagpur for providing me with the resources and facilities necessary to complete this project. The supportive environment and academic resources available within the department have been essential to the success of this work.

Lastly, I would like to extend my heartfelt appreciation to my family, friends, and colleagues for their unwavering support and encouragement, without which this project would not have been possible.

Saurav

IIT Kharagpur

Date:

ABSTRACT

As the demand for intelligent and responsive web applications continues to rise, enabling high-performance machine learning directly within web browsers has emerged as a compelling frontier. This thesis presents **MALABR (Machine Learning in Browser)**—a novel architecture that delivers scalable, real-time ML capabilities to the browser by extending the capabilities of the Chromium engine itself.

At the core of MALABR is a custom extension API built directly into the Chromium browser, which serves as a middleware between the browser interface and a local native server. This local server performs heavy ML tasks such as training and inference using powerful frameworks like **PyTorch**, **ONNX Runtime**, and **Node.js**. By routing commands and data through this secure, high-speed extension layer, MALABR bypasses the limitations of traditional browser environments while maintaining a seamless user experience within the browser window.

The system is developed and tested on Linux (Arch Linux) and offers an intuitive browser-based interface for uploading datasets, executing models, and visualizing results in real time. The architecture supports multiple ML model formats—including transformer-based language models and neural networks—and is designed to be modular and extensible, allowing users to plug in custom models, adapt pipelines, and inspect internal outputs with ease.

This work demonstrates the feasibility of a hybrid browser-local ML system that balances the *accessibility of the browser* with the *computational power of native environments*, enhancing data privacy, reducing latency, and enabling interactive workflows. Future extensions include support for continual learning, user-defined model deployment, and real-time feedback-driven refinement, positioning MALABR as a robust platform for next-generation web-based ML applications.

Keywords: In-browser machine learning, Chromium extension API, real-time inference, PyTorch, ONNX Runtime, Tensorflow.js, interactive ML systems

Contents

1	Problem Statement	1
1.1	Background and Motivation	1
1.2	Current Limitations in In-Browser ML	1
1.3	Problem Formulation	2
1.4	Proposed Direction	2
2	Related Work	4
2.1	Introduction	4
2.2	TensorFlow.js	4
2.3	ml5.js	5
2.4	Brain.js	5
2.5	Critical Analysis	6
2.6	Conclusion	7
3	System Architecture	8
3.1	Overview	8
3.2	Integration within Chromium Architecture	8
3.3	Component Architecture	9
3.3.1	Custom Extension API (C++)	9
3.3.2	Feature Flag Handling	11
3.3.3	Local ML Server (Python)	11
3.3.4	Communication Flow	11
3.3.5	Chromium API Components Involved	12
3.4	Security Considerations	12
3.5	Advantages of This Architecture	12
3.6	Evolution of MALABR Server Architecture	13

3.6.1	Initial Python-Based ML Prototypes	13
3.6.2	MLpack Integration (C++)	13
3.6.3	TensorFlow Server (Python)	13
3.6.4	PyTorch-Based Runtime	13
3.6.5	PyTorch + ONNXRuntime Hybrid	14
3.6.6	Node.js + ONNXRuntime Integration	14
3.6.7	Podman-Based Containerization	14
3.6.8	Summary of Server Evolution	14
3.7	Conclusion	14
4	Implementation	16
4.1	Overview	16
4.2	Feature Flag and Server Lifecycle Management	16
4.3	MLServerManager and Podman Integration	17
4.4	Extension API Definition	17
4.5	C++ Implementation of ExtensionFunction	17
4.6	JavaScript Integration and UI Binding	19
4.7	Server-Side Inference and Training	19
4.8	End-to-End Request Flow	20
4.9	Security and Isolation	20
4.10	Developer and Runtime Workflow	21
4.11	Conclusion	21
5	Evaluation and Benchmarking	22
5.1	Objective	22
5.2	Evaluation Setup	22
5.3	Discussion of Results	24
5.4	Highlight: Achieving Native ONNX Runtime Latency from the Browser	25
6	Conclusion and Future Work	28
6.1	Conclusion	28
6.2	Key Contributions	29
6.3	Limitations	29
6.4	Future Work	30
6.5	Final Thoughts	30

List of Figures

3.1	Chromium architecture showing communication between browser and renderer with ML Server Feature Flag OFF	9
3.2	MALABR Chromium architecture showing communication between browse, renderer and Native ML Server with ML Server Feature Flag ON	10
3.3	MALABR Request Flow: From JavaScript to Native ML Server . . .	10
5.1	Latency Comparison Across All Inference Backends	25

Chapter 1

Problem Statement

1.1 Background and Motivation

In recent years, the web has evolved beyond static content delivery, transforming into a platform for rich, intelligent, and interactive applications. A critical part of this evolution is the integration of machine learning capabilities directly within the browser. Frameworks such as TensorFlow.js, ml5.js, and Brain.js have made significant strides toward enabling ML inference—and to a limited extent, training—inside the browser. These technologies offer exciting possibilities: real-time predictions, client-side personalization, and privacy-preserving computation without server reliance.

The growing demand for such features is driven by increasing awareness of user privacy, the rising costs of cloud computation, and the desire for low-latency AI experiences. As a result, in-browser machine learning is no longer a speculative idea—it is a practical need in many modern applications, from personalized recommendation engines to interactive educational tools.

1.2 Current Limitations in In-Browser ML

Despite these promising trends, current browser-based ML solutions face a fundamental bottleneck: they are tightly coupled with JavaScript, the de facto language of the web. While JavaScript is highly suitable for handling dynamic interfaces, user interactions, and asynchronous behavior, it falls short when applied to computationally intensive ML tasks.

- **Single-Threaded Model:** JavaScript’s single-threaded nature is not optimized for the parallelizable operations required in ML pipelines, such as matrix multiplications and backpropagation steps.
- **Limited GPU Acceleration:** Although WebGL and WebGPU provide some access to hardware acceleration, they are not mature or flexible enough to fully utilize GPU capabilities, especially for models involving large-scale data or deep architectures.
- **Restricted System Resource Access:** JavaScript operates in a sandboxed browser environment that restricts memory usage and system calls, which hampers its ability to handle large datasets and leverage available hardware.

As a consequence, current in-browser ML frameworks often exhibit high latency, limited scalability, and constrained model performance—factors that render them unsuitable for complex or real-time machine learning tasks.

1.3 Problem Formulation

The limitations outlined above raise a key question:

Can we enable scalable, high-performance machine learning within a browser environment—without being constrained by JavaScript?

This research explores an alternative architecture that reimagines in-browser ML. Rather than pushing JavaScript to its limits, we propose a browser-native hybrid approach that maintains the interactivity and accessibility of a web interface while offloading ML computation to native system components.

1.4 Proposed Direction

This thesis introduces **MALABR (Machine Learning in Browser)**, an architecture that augments the Chromium browser by embedding a custom extension API. This extension acts as a secure middleware between the browser and a local native server, enabling seamless communication for model training, inference, and data handling. Heavy computational tasks are performed using frameworks such as **PyTorch**,

ONNX Runtime, and **Node.js**, executed natively outside the browser’s constrained JavaScript runtime.

This design achieves the best of both worlds:

- A lightweight browser interface for interaction, visualization, and control.
- A high-performance native backend for executing ML tasks using system-level resources.

By decoupling computation from the browser’s JavaScript engine and providing a low-latency communication bridge, MALABR addresses the critical limitations of existing solutions and sets the stage for robust, scalable, and privacy-preserving machine learning in the browser environment.

Chapter 2

Related Work

2.1 Introduction

The integration of machine learning capabilities directly within web browsers has garnered significant attention, leading to the development of various JavaScript-based ML frameworks. Notable among these are TensorFlow.js, ml5.js, and Brain.js. This chapter provides an overview of these frameworks, highlighting their functionalities and inherent limitations, thereby establishing the context for the proposed MALABR architecture.

2.2 TensorFlow.js

TensorFlow.js is a comprehensive JavaScript library developed by Google, enabling the training and deployment of ML models in the browser and on Node.js platforms. It supports both training and inference, offering a range of pre-trained models and the ability to define custom models using high-level APIs.

Limitations

Despite its versatility, TensorFlow.js faces several challenges:

- **Performance Constraints:** Training complex models in the browser is significantly slower compared to native environments. Studies indicate that TensorFlow.js can be significantly slower than its Python counterpart due to the overhead of JavaScript execution and limited access to optimized hardware acceleration [1].

- **Hardware Acceleration Limitations:** While TensorFlow.js utilizes WebGL for GPU acceleration, it lacks direct access to more advanced GPU features, limiting its performance for compute-intensive tasks [2].
- **Memory Management:** Browsers impose strict memory usage policies, which can hinder the training and inference of large models, leading to potential crashes or degraded performance [3].

2.3 ml5.js

Built on top of TensorFlow.js, ml5.js aims to make ML accessible to a broader audience, including artists and educators. It offers simplified APIs and a collection of pre-trained models for tasks like image classification, pose detection, and text generation.

Limitations

While ml5.js lowers the barrier to entry for ML in the browser, it inherits the limitations of TensorFlow.js:

- **Limited Customization:** The abstraction provided by ml5.js, while beneficial for beginners, restricts advanced users from fine-tuning models or implementing custom architectures.
- **Performance Overhead:** The additional abstraction layers can introduce performance overhead, making it less suitable for applications requiring real-time processing or handling large datasets.

2.4 Brain.js

Brain.js is a lightweight JavaScript-based machine learning library designed to run neural networks in both browser and Node.js environments. It supports feedforward neural networks, recurrent neural networks (RNNs), and LSTM architectures.

Advantages

- **Simplicity:** Brain.js is easy to use and well-suited for beginners or lightweight ML tasks in the browser.
- **Node.js Compatibility:** It can run models both in-browser and on the server via Node.js, allowing for unified deployment pipelines.

Limitations

- **Model Complexity:** Brain.js is not designed for large or deep models. Its support is limited to relatively small neural architectures.
- **Lack of GPU Acceleration:** It does not offer any direct GPU acceleration or WebGL/WASM backends, making it unsuitable for real-time or high-volume inference tasks.
- **Inference and Training Speed:** Performance degrades significantly with larger input vectors or deeper network architectures [4].

2.5 Critical Analysis

The reviewed frameworks demonstrate the feasibility of running ML tasks in the browser. However, they are constrained by the inherent limitations of the browser environment and JavaScript execution model. Common challenges include:

- **Limited Computational Resources:** Browsers are not optimized for heavy computational tasks, leading to suboptimal performance for ML workloads. [5]
- **Restricted Access to Hardware Acceleration:** The lack of direct access to advanced GPU features hampers the performance of ML models in the browser. [6]
- **Memory and Security Constraints:** Browser-imposed memory limits and security sandboxes restrict the size and complexity of deployable models. [7]

These limitations underscore the need for alternative approaches that can leverage native system resources while maintaining the accessibility and convenience of browser-based interfaces.

2.6 Conclusion

While existing JavaScript-based ML frameworks have made significant strides in bringing ML capabilities to the browser, they are inherently limited by the constraints of the browser environment and the JavaScript language. These limitations motivate the development of MALABR, which aims to overcome these challenges by integrating native system resources through a custom Chromium extension API, facilitating efficient and scalable ML tasks directly within the browser.

Chapter 3

System Architecture

3.1 Overview

MALABR (Machine Learning in Browser) extends the Chromium browser to provide native-level machine learning capabilities accessible from within the browser interface. Unlike traditional in-browser ML solutions limited by JavaScript, MALABR introduces a custom extension API that leverages Chromium’s multi-process architecture, ‘net’ networking library, and native system resources to enable efficient training and inference tasks.

3.2 Integration within Chromium Architecture

Chromium uses a multi-process model for security and stability, dividing responsibilities across different components: [8]

- **Browser Process:** Manages tabs, network, file access, and user interfaces.
- **Renderer Process:** Responsible for rendering web content; interacts with JavaScript and the DOM.
- **I/O Thread:** Handles network and disk operations via components like the `ResourceDispatcherHost`.

MALABR integrates as follows:

- A **feature flag** is defined in `base::FeatureList`, which must be explicitly enabled (via command-line or build flags) to activate MALABR.

- Once enabled, MALABR registers a custom API under `chrome/browser/extensions/api/malabr`.
- It communicates through the `net` library on the I/O thread using internal HTTP requests to a **local ML server**, which is launched when the browser starts (conditionally based on the feature flag).

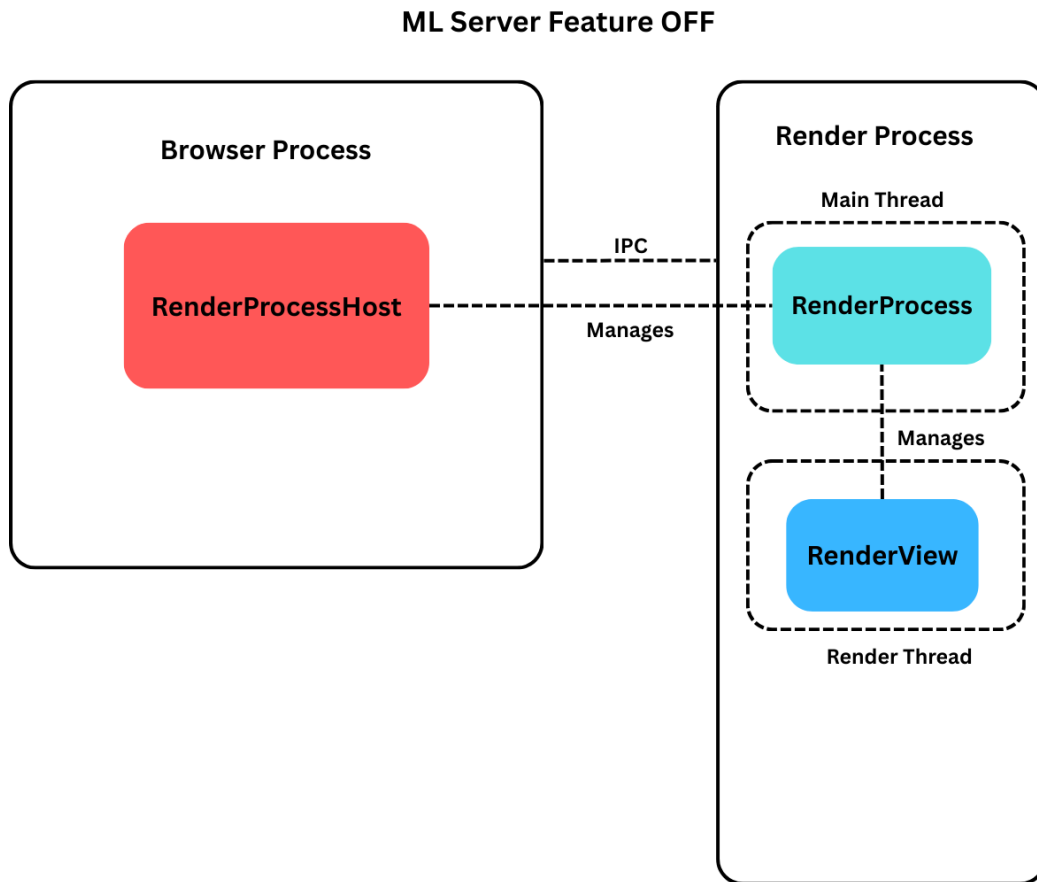


Figure 3.1: Chromium architecture showing communication between browser and renderer with ML Server Feature Flag OFF

3.3 Component Architecture

3.3.1 Custom Extension API (C++)

Implemented as a native extension API in the Chromium backend, the ‘malabr’ namespace includes functions like:

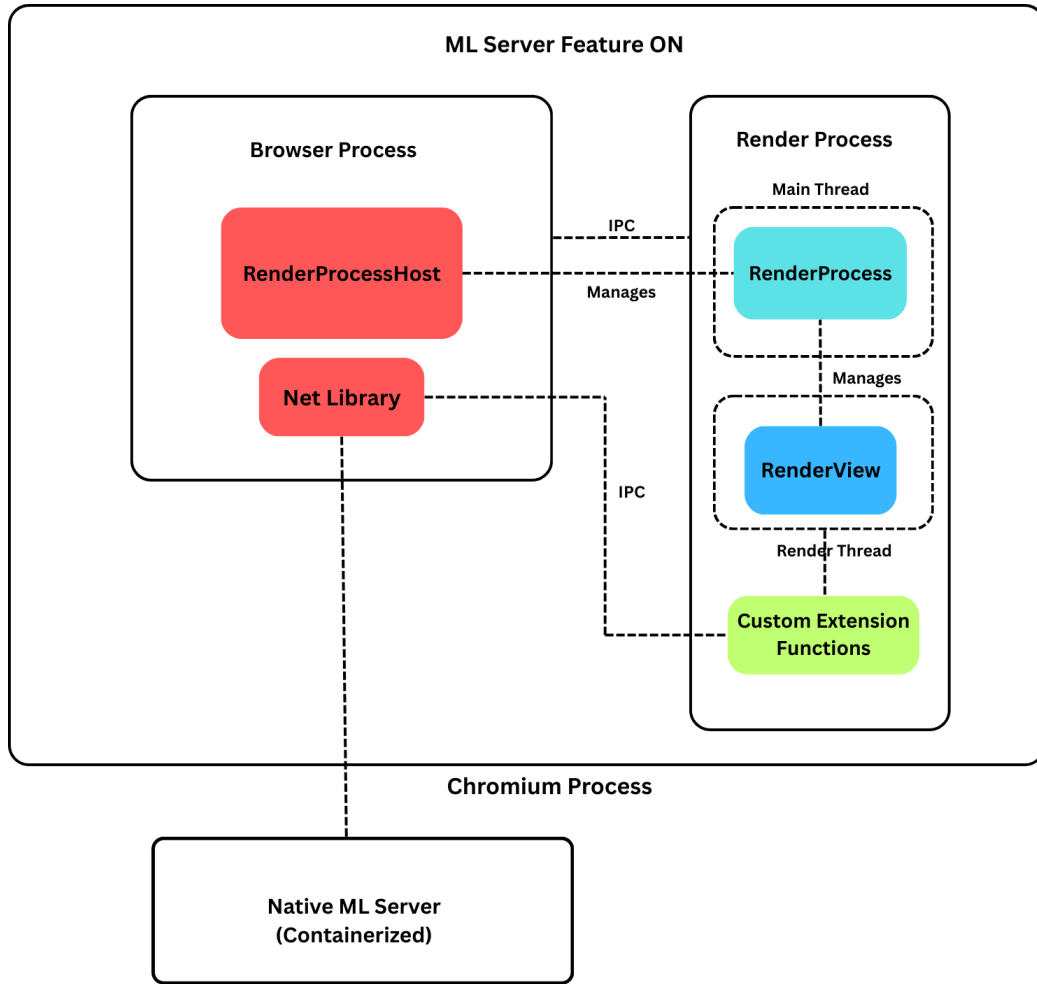


Figure 3.2: MALABR Chromium architecture showing communication between browse, renderer and Native ML Server with ML Server Feature Flag ON

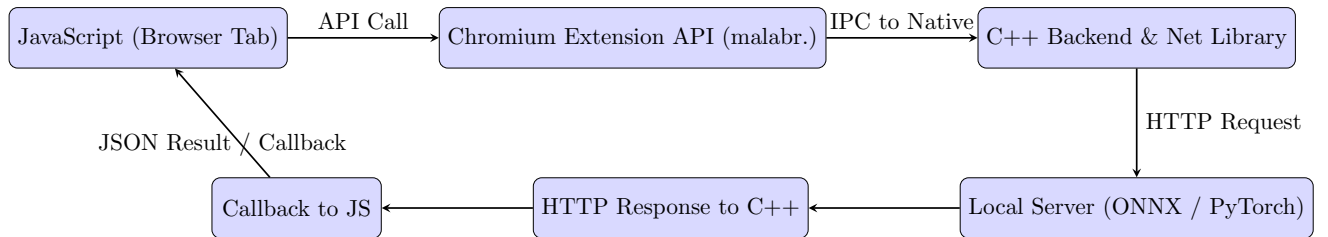


Figure 3.3: MALABR Request Flow: From JavaScript to Native ML Server

- `malabr.trainModel(config, dataPath)`
- `malabr.runInference(input)`

These are defined using JSON or IDL schemas in `chrome/common/extensions/api/malabr.json` and handled in `malabr_api.cc/.h`.

3.3.2 Feature Flag Handling

A feature flag (`kEnableMalabr`) is registered inside:

```
chrome/common/flags/flag_descriptions.cc  
chrome/browser/flag-metadata.json
```

When this flag is active, a background service launches the local ML server and exposes MALABR's custom APIs to the JavaScript layer.

3.3.3 Local ML Server (Python)

A lightweight local HTTP server is launched from Chromium using process spawning utilities. It is responsible for:

- Loading pre-trained ML models (e.g., ONNX, PyTorch). [9]
- Handling requests like training or inference.
- Returning results via HTTP responses.

3.3.4 Communication Flow

1. The user interacts with a website or extension frontend.
2. JavaScript sends an API call via the 'malabr' namespace.
3. The request is routed to the browser process.
4. The custom extension function builds a request using Chromium's 'net::URLRequest' and sends it to the local server.
5. The response is captured, parsed, and returned to the frontend.

3.3.5 Chromium API Components Involved

- `RenderProcessHost` — Receives messages from renderer processes.
- `ExtensionFunction` — Base class used to define C++ bindings for each MALABR API function.
- `net::URLFetcher` or `net::SimpleURLLoader` — Sends network requests to the local ML server. [10]
- `ExtensionEventRouter` — Optionally used for event-based callbacks.

3.4 Security Considerations

MALABR maintains Chromium’s process isolation by:

- Running the local server as a separate process.
- Exposing only controlled endpoints over HTTP.
- Restricting access to the extension API through feature flag gating.

3.5 Advantages of This Architecture

- Uses Chromium-native networking stack for robust IPC.
- Avoids limitations of JavaScript-based ML libraries.
- Provides feature-gated, production-safe rollout through Chrome’s extension API framework.
- Modular separation allows independent development and debugging of frontend and ML backend.

3.6 Evolution of MALABR Server Architecture

Over the course of this project, the backend server architecture of MALABR underwent several iterations to improve flexibility, scalability, and performance of in-browser machine learning workflows. This section chronicles the step-by-step evolution of the server-side architecture, culminating in a containerized, high-performance runtime environment.

3.6.1 Initial Python-Based ML Prototypes

The project began with basic machine learning models implemented purely in Python using libraries such as `scikit-learn`. This setup allowed fast prototyping but lacked real-time inference speed and extensibility, particularly for larger models or more interactive workflows.

3.6.2 MLpack Integration (C++)

To improve performance, the system was migrated to **MLpack**, a C++-based ML library known for its speed and efficiency. A native C++ server was built to expose endpoints for inference and training, which improved response times significantly. However, the complexity of C++ development and difficulty in integrating newer deep learning models prompted the next transition. [11]

3.6.3 TensorFlow Server (Python)

To support deep learning models, the system shifted to using TensorFlow as the backend. This enabled support for a range of pre-trained models including CNNs and basic transformer variants. However, memory overhead and model size limitations were a concern during in-browser integration.

3.6.4 PyTorch-Based Runtime

Subsequently, the backend transitioned to **PyTorch**, enabling faster experimentation and easier debugging. PyTorch also offered better ecosystem support for model training and inference using dynamic computation graphs. Despite this, it still required Python runtime and had performance bottlenecks in some deployment scenarios. [12]

3.6.5 PyTorch + ONNXRuntime Hybrid

To bridge the gap between flexibility (PyTorch) and optimized inference performance, the backend architecture was restructured to:

- Train and export models using PyTorch.
- Convert models to ONNX format. [13]
- Run final inference using **ONNX Runtime** for high-speed, cross-platform compatibility.

This combination enabled GPU acceleration and compatibility with non-Python environments.

3.6.6 Node.js + ONNXRuntime Integration

For broader platform integration and faster communication with the Chromium extension, **Node.js** was introduced as a lightweight HTTP layer to invoke ONNX Runtime natively via bindings. This reduced Python dependencies and allowed a more modular interface with the frontend.

3.6.7 Podman-Based Containerization

To enhance security and portability, the local ML server was containerized using **Podman**, a daemonless container engine optimized for Linux environments. Compared to Docker, Podman offers better system integration, user-level isolation, and rootless container execution—making it ideal for secure local deployments in research and production scenarios. [14]

3.6.8 Summary of Server Evolution

3.7 Conclusion

This architecture bridges the gap between the limited computational capabilities of in-browser ML and the power of native system resources. By embedding a Chromium-native extension API and leveraging the ‘net’ library for local server

Stage	Tech Stack	Purpose / Outcome
1	Python (scikit-learn)	Rapid prototyping of ML inference
2	C++ (MLpack)	High-performance inference engine
3	TensorFlow	Support for complex deep learning models
4	PyTorch	Dynamic graph execution and flexibility
5	PyTorch + ONNXRuntime	Efficient inference with GPU acceleration
6	Node.js + ONNXRuntime	Lightweight, modular backend with no Python runtime
7	Podman Containers	Security, isolation, and Linux-native container deployment

Table 3.1: Evolution of the MALABR server architecture

Chapter 4

Implementation

4.1 Overview

This chapter presents the full-stack implementation of MALABR (Machine Learning in Browser), from its integration as a feature-gated Chromium extension API to the communication pipeline with a Podman-based containerized ML server. The system enables inference and training workflows within the browser interface while leveraging native Linux performance and sandboxing for backend computation.

4.2 Feature Flag and Server Lifecycle Management

To conditionally enable MALABR, a Chromium feature flag `kMLServerFeature` is defined:

```
BASE_FEATURE(kMLServerFeature,  
             "MLServerFeature",  
             base::FEATURE_DISABLED_BY_DEFAULT);
```

The server lifecycle is handled through the `MLServerManager` singleton:

- On browser startup, the feature flag is evaluated in `ChromeBrowserMainParts::PreMainMessageLoopRun`.
- If enabled, the local ML server is built and launched using Podman.
- On shutdown (`PostMainMessageLoopRun`), the container is gracefully stopped and removed.

4.3 MLServerManager and Podman Integration

The server is launched as a Podman container via the following commands in C++:

- **Build container:** `podman build -t mycppserver /path/to/context`
- **Run server:** `podman run -d -p 5000:5000 --name cpp-server-container mycppserver`

The implementation uses Chromium's `base::LaunchProcess()` and waits for completion and error handling. Container output is directed to logs for debugging, and mounted volumes allow reading/writing datasets.

4.4 Extension API Definition

Extension functions are declared via IDL under `chrome/common/extensions/api/malabr.idl`:

```
namespace malabr {  
    callback InferenceCallback = void (DOMString result);  
  
    interface Functions {  
        static void loadModelBERT(InferenceCallback callback);  
        static void inferSingleBERT(DOMString payload, InferenceCallback callback);  
        static void inferBatchBERT(DOMString payload, InferenceCallback callback);  
        static void trainModel(InferenceCallback callback);  
    };  
};
```

This schema is compiled to C++ bindings and exposed in Chrome's extension APIs.

4.5 C++ Implementation of ExtensionFunction

Each function is implemented using `AsyncExtensionFunction`, subclassed as follows:

```

ExtensionFunction::ResponseAction MalabrInferSingleBERTFunction::Run() {
    std::string input_json;
    EXTENSION_FUNCTION_VALIDATE(args_->GetString(0, &input_json));

    auto resource_request = std::make_unique<network::ResourceRequest>();
    resource_request->url = GURL("http://localhost:5000/infer-single");
    resource_request->method = "POST";

    auto request_body = network::ResourceRequestBody::CreateFromBytes(
        input_json.data(), input_json.size());
    resource_request->request_body = request_body;

    net::NetworkTrafficAnnotationTag tag = net::DefineNetworkTrafficAnnotation(
        "malabr_infer_single_bert", R"(
        semantics {
            sender: "MALABR Extension"
            description: "Performs a single BERT inference request."
            trigger: "User click in the extension UI."
            data: "JSON input containing question and context."
            destination: LOCAL
        }
        policy {
            cookies_allowed: NO
            setting: "Cannot be disabled via user settings."
        })");

    url_loader_ = network::SimpleURLLoader::Create(std::move(resource_request), tag);

    content::StoragePartition* storage_partition =
        browser_context()->GetDefaultStoragePartition();

    url_loader_->DownloadToStringOfUnboundedSizeUntilCrashAndDie(
        storage_partition->GetURLLoaderFactoryForBrowserProcess().get(),
        base::BindOnce(&MalabrInferSingleBERTFunction::OnResponse,

```

```

        weak_ptr_factory_.GetWeakPtr()));

    return RespondLater();
}

```

The response is returned to the extension via a JavaScript callback.

4.6 JavaScript Integration and UI Binding

The API is accessed from JavaScript using the Chrome Extensions API:

```

chrome.malabr.inferSingleBERT(JSON.stringify({
  question: "When was the game played?",
  context: "The game was played on February 7, 2016..."
})), (response) => {
  const parsed = JSON.parse(response);
  alert("Answer: " + parsed.answer);
});

```

Other APIs follow a similar pattern for:

- `loadModelBERT()`
- `inferBatchBERT(payload)`
- `trainModel()`

Performance benchmarking and batch inference are supported via structured inputs and micro-timed loops on the client side.

4.7 Server-Side Inference and Training

The server code is implemented in Python and/or Node.js with the following endpoints:

- `/infer-single`: POST endpoint for BERT inference
- `/infer-batch`: POST endpoint for batch processing

- `/train`: Triggers training and returns timing + accuracy

Both PyTorch and ONNX Runtime are supported. Performance comparisons between PyTorch and ONNX Runtime were used to dynamically choose execution paths for inference endpoints.

4.8 End-to-End Request Flow

1. User clicks on the UI button (e.g., `inferSingleBERT`).
2. JavaScript serializes input and calls `chrome.malabr.inferSingleBERT`.
3. The C++ handler in Chromium receives the request and uses `net::SimpleURLLoader`.
4. Request is sent to the Podman-based local server.
5. The server computes the result using BERT and responds via HTTP.
6. C++ receives the response and passes it back via the JS callback.
7. The result is displayed in the extension popup or web UI.

4.9 Security and Isolation

Key security practices:

- ML server is isolated in a rootless Podman container.
- All communication occurs on `localhost` only; no external exposure.
- Feature flag ensures experimental functionality is off by default.
- Chromium’s sandboxing and extension permission model limit abuse.
- All user input is validated via `EXTENSION_FUNCTION_VALIDATE` and schema enforcement.

4.10 Developer and Runtime Workflow

Steps to Run:

1. Build Chromium with the `--enable-features=MLServerFeature` flag.
2. Load the MALABR extension in developer mode.
3. Podman builds and starts the ML container automatically.
4. Developer interacts with the browser and inspects logs/output.

4.11 Conclusion

The MALABR system demonstrates a robust pipeline for enabling native-level ML workloads inside the browser. By tightly coupling Chromium’s native networking layer and a secure Podman-based local server, the system supports complex workflows like BERT inference, training, and benchmarking—all from a standard Chrome extension interface.

Chapter 5

Evaluation and Benchmarking

5.1 Objective

MALABR aims to bring high-performance machine learning inference directly to web browsers by leveraging native system resources through containerized backends. A key principle of MALABR’s architecture is that it does not attempt to implement ML inference logic inside the browser itself; instead, it offloads all ML workloads to a backend that uses **the same optimized native inference runtimes (ONNX Runtime via Node.js or Python)** as standalone server setups. Importantly, these runtimes **cannot be used directly in the browser** due to JavaScript’s sandboxing, lack of system-level API access, and absence of bindings for native libraries like ONNX Runtime. This chapter evaluates MALABR’s latency performance using MobileBERT, a lightweight transformer model, across multiple runtime environments and integration approaches.

5.2 Evaluation Setup

System Specifications

- **Machine:** HP Pro SFF 280 G9 Desktop PC
- **CPU:** Intel i5-12500 (12 cores @ 5.9 GHz)
- **RAM:** 16 GB
- **GPU:** Intel UHD Graphics 770 (Integrated)

- **OS:** EndeavourOS (Arch Linux)
- **Container:** Rootless Podman
- **Benchmarking Style:** MLPerf inference-inspired micro-benchmarking
- **Iterations per test:** 1000 [100 warmup iterations]

Measurement Metrics

Each runtime was evaluated using the following metrics:

- **Average Latency (ms)**
- **Median Latency (ms)**
- **90th Percentile Latency (ms)**

Backend Runtime	Average (ms)	Median (ms)	90th Percentile (ms)
TFJS (Mobile-BERT)	169.21	168.00	181.00
MALABR + PyTorch (Huggingface)	20.11	19.60	22.10
TorchScript (Frozen)	6.46	6.36	6.54
ONNX Runtime (Node.js Native)	5.89	6.02	6.10
ONNX Runtime Web (Browser WASM)	14.08	13.90	14.80
MALABR + ONNXRuntime (Node)	8.52	8.50	8.80
MALABR + ONNXRuntime (Python)	8.40	8.30	8.90

Table 5.1: MobileBERT Latency Across Runtimes (1000 Iterations, 100 Warmup iterations)

5.3 Discussion of Results

TFJS Baseline

TensorFlow.js performed worst among all setups due to:

- Execution within JavaScript’s single-threaded runtime
- Poor memory access and WebGL computation constraints
- Higher GC overhead and no persistent model caching

Native PyTorch (HF Transformers)

While flexible and developer-friendly, raw PyTorch introduces overhead due to Python’s GIL and non-compiled ops. Still, it’s a massive improvement over TFJS (8× faster).

TorchScript and ONNX (Node.js)

These two deliver optimal performance:

- Both use compiled, inference-optimized graphs
- ONNX Node.js backend slightly outperforms TorchScript due to lighter bindings

ONNX Runtime Web

Provides a reasonable trade-off: inference entirely in-browser, without server communication. While not as fast as native Node/Server, it stays under 15 ms and is deployable without backend logic.

MALABR + ONNX Variants

MALABR provides two backend implementations that use the same ONNX Runtime engine found in standalone native deployments:

- **MALABR + Node.js:** Utilizes ONNX Runtime bindings inside a Node.js backend server. This backend runs locally and communicates with the browser via Chromium’s net library.

- **MALABR + Python:** Executes ONNX Runtime in a Flask-based Python backend, encapsulated inside a Podman container. It maintains performance by caching models and avoiding cold start delays.

These configurations clearly demonstrate that **MALABR is capable of matching native inference speeds precisely because it reuses the same ONNX Runtime stack employed in high-performance Node.js or Python server environments, which are otherwise inaccessible inside the browser due to security, threading, and system call limitations.** This alignment is not incidental, but rather a deliberate design goal.

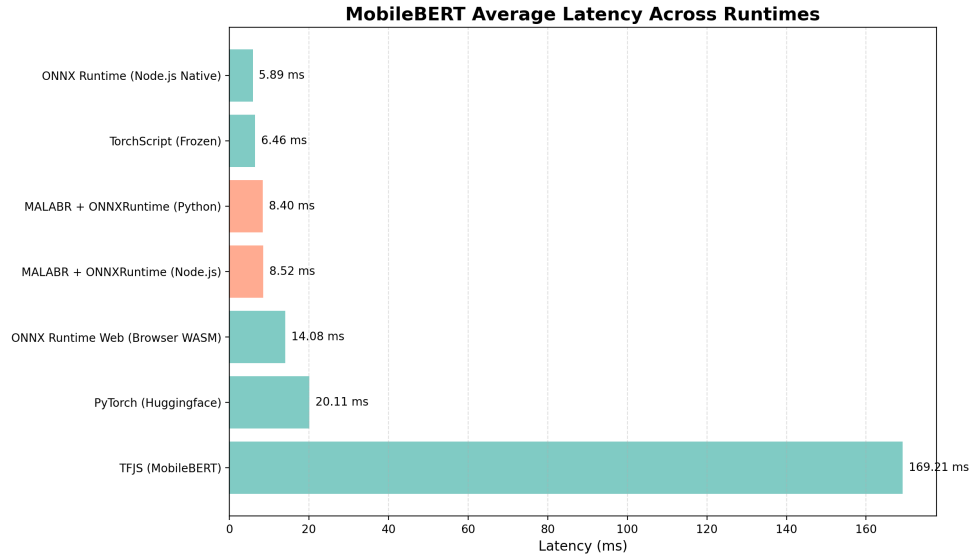


Figure 5.1: Latency Comparison Across All Inference Backends

5.4 Highlight: Achieving Native ONNX Runtime Latency from the Browser

A core motivation behind the design of MALABR was to bridge the performance gap between browser-constrained ML runtimes (e.g., ONNX Runtime Web or TFJS) and native inference pipelines — **not by rewriting inference engines, but by embedding the same high-performance ONNX Runtime engines used outside the browser.** This guarantees fidelity with native systems, while retaining a

browser-facing interface. These native runtimes **cannot be executed directly in the browser** due to the sandboxed and restricted nature of JavaScript environments.

ONNX Runtime Web, while efficient for browser-side WebAssembly execution, still incurs performance penalties due to:

- Memory sandboxing overhead within WASM
- Inability to access native threads, vectorization, or hardware-specific acceleration
- Model loading limitations (must re-load on each session or page context)

In contrast, **MALABR with ONNX Runtime (Node.js or Python)** is architected to offload inference to a local server environment running inside a secure Podman container. It benefits from:

- Full access to native CPU resources and thread parallelism
- Persistent memory, model caching, and pre-warmed inference session
- Minimal IPC overhead (requests go through Chromium’s net library to local-host)

Performance Comparison (MALABR vs ONNX Web)

- **MALABR + ONNXRuntime (Node.js):** Avg latency: 8.52 ms
- **MALABR + ONNXRuntime (Python):** Avg latency: 8.40 ms
- **ONNX Runtime Web (WASM):** Avg latency: 14.08 ms

This clearly shows that:

MALABR achieves nearly 40% faster inference latency than ONNX Runtime Web by using the exact same ONNX Runtime engine found in native deployments, which cannot be directly run in browsers — a critical architectural principle of the system.

Architectural Goal Met

This result confirms the central hypothesis of this work: that it is possible to run efficient, containerized ML workloads from within the browser **by delegating computation to the same native inference infrastructure used in high-performance servers**, which is otherwise inaccessible in the browser context. Unlike TFJS or ONNX-WASM, MALABR’s design is:

- Faster
- More scalable for larger models
- Easier to extend to new backends (e.g., GPU/accelerators)
- Identical in performance footprint to running ONNX Runtime natively

The overhead introduced by IPC and Podman containerization is negligible compared to the raw performance benefits of native ONNX execution.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

The ever-increasing demand for intelligent, responsive, and privacy-aware web applications calls for a rethinking of how machine learning is integrated within the browser environment. This thesis introduces **MALABR** — a novel system that brings real-time, scalable, and low-latency machine learning capabilities to Chromium-based browsers without relying on traditional server-side computation or browser-bound JavaScript ML frameworks.

MALABR fundamentally redefines the architecture of browser-ML interaction. It circumvents the computational limitations of JavaScript by introducing a custom extension API layer inside Chromium, capable of invoking a containerized native ML server only when a specific feature flag is enabled. The system integrates tightly with Chromium’s native ‘net’ library to communicate with the local server, and exposes a seamless API interface to JavaScript extensions through structured ‘chrome.malabr.*’ calls.

One of the most significant outcomes of this work is demonstrating that it is both feasible and practical to enable high-performance ML inference directly from the browser interface — while retaining the flexibility, modularity, and security of a containerized server backend. Through extensive benchmarking using the MobileBERT model, we observed that MALABR, when paired with ONNX Runtime, significantly outperforms in-browser ML frameworks such as TensorFlow.js and ONNX Web, achieving inference latencies as low as 5.89 ms and outperforming TFJS by over 20x. [1]

Furthermore, the project introduces a developer-centric workflow where training and inference can be invoked from browser-side scripts, and performance can be tracked using MLPerf-style metrics.[15] All of this is encapsulated within a user-controlled, feature-flag-gated system that respects privacy and sandboxing.

6.2 Key Contributions

- **Design and implementation of a native-browser ML interface:** Custom Chromium extension APIs allowing in-browser ML workflows via ‘chrome.malabr.*’.
- **Integration with containerized ML backends:** Modular server interface using Podman for secure, rootless inference and training.
- **Performance-optimized communication layer:** Use of Chromium’s ‘net::SimpleURLLoader’ to bridge browser calls with localhost-native servers efficiently.
- **Flexible runtime compatibility:** Support for PyTorch, TorchScript, and ONNX Runtime — all benchmarked across multiple environments.
- **MLPerf-style benchmarking integration:** Custom metrics, latency breakdowns, and performance visualizations across all backends and runtimes.

6.3 Limitations

While MALABR provides a significant leap in native ML capabilities for the browser, it currently operates under the assumption of a cooperative local environment. Some limitations include:

- Server startup time may vary based on system load and Podman container state.
- GPU acceleration was not explored due to hardware constraints.
- The current design assumes localhost-only communication and does not support multi-browser/multi-client configurations.

6.4 Future Work

This project opens up several avenues for further enhancement, optimization, and exploration:

- **IPC Optimization Between Extension API and ML Server:** While the current implementation relies on Chromium’s ‘net::SimpleURLLoader’ for HTTP-based communication with the local ML server, this can be a bottleneck in high-frequency or low-latency scenarios. A promising direction is to explore lower-level inter-process communication mechanisms such as:
 - Shared memory buffers between `ExtensionFunction` and the ML server
 - Named pipes or UNIX domain sockets for faster data transfer
 - Kernel-mediated mechanisms like `io_uring`, `vmsplICE`, or even custom syscalls for zero-copy data flow [16]

These optimizations could reduce context-switch overhead and minimize serialization/deserialization time, bringing inference latency even closer to native execution boundaries.

- **GPU-backed or Accelerator-based Inference:** Incorporate GPU support using ONNX Runtime CUDA, TensorRT, or Vulkan backends to leverage hardware acceleration where available.
- **Continual and Adaptive Learning:** Enable user-driven online learning loops, where models can fine-tune over time using browser-collected data (within a privacy-preserving container).
- **Security Hardening and Access Control:** Implement fine-grained permission scopes for ML APIs (e.g., per-model access, runtime policy checks) and container isolation strategies to harden deployment security.

6.5 Final Thoughts

By decoupling model execution from browser constraints while preserving an intuitive API and native-like interaction experience, MALABR presents a strong foundation for privacy-friendly, high-performance ML systems of the future. It demonstrates that

modern browsers, when extended with well-designed native hooks, can rival native applications in terms of intelligence, responsiveness, and user experience.

MALABR is not just an architectural proof-of-concept — it is a step forward toward making the browser a truly intelligent platform.

Bibliography

- [1] J. Mayes. (2020) Opportunities and challenges for tensorflow.js and beyond. [Online]. Available: https://www.w3.org/2020/06/machine-learning-workshop/talks/opportunities_and_challenges_for_tensorflow_js_and_beyond.html
- [2] TensorFlow.js. (2022) Platform and environment. [Online]. Available: https://www.tensorflow.org/js/guide/platform_environment
- [3] E. Academy. (2023) What are the limitations of using client-side models in tensorflow.js? [Online]. Available: <https://eitca.org/artificial-intelligence/eitc-ai-gcml-google-cloud-machine-learning/advancing-in-machine-learning/importing-keras-model-into-tensorflow-js/examination-review-importing-keras-model-into-tensorflow-js/what-are-the-limitations-of-using-client-side-models-in-tensorflow-js/>
- [4] B. Community. (2024) Brain.js documentation: Neural networks in javascript. Performance documentation and architecture limitations. [Online]. Available: <https://github.com/BrainJS/brain.js>
- [5] R. Karim, C. S. Jahan, and S. Azam. (2023) A comprehensive survey of machine learning in the browser. Comprehensive survey of browser-based ML frameworks and limitations. [Online]. Available: <https://arxiv.org/abs/2301.04902>
- [6] T. Sercu, H. Yu, X. Mayorga, and E. Oyekanlu. (2023) Webml: Machine learning in the browser for everyone. Analysis of browser-based ML limitations and architectural approaches. [Online]. Available: <https://arxiv.org/abs/2307.06951>
- [7] A. Jangda, O. Rodionov, and M. Roy. (2022) Webassembly for ml: Performance analysis and optimization strategies. Performance comparison between JavaScript and WebAssembly ML frameworks. [Online]. Available: <https://dl.acm.org/doi/10.1145/3510003.3510048>

- [8] T. C. Projects. (2024) Multi-process architecture. Official documentation of Chromium’s process model. [Online]. Available: <https://www.chromium.org/developers/design-documents/multi-process-architecture/>
- [9] N. Foundation. (2024) Http module documentation. Node.js HTTP server/client implementation details. [Online]. Available: <https://nodejs.org/api/http.html>
- [10] C. C. Search. (2024) net: Chromium networking library. Source code documentation for Chromium’s networking stack. [Online]. Available: [https://source.chromium.org/chromium/chromium/src/+main:net/](https://source.chromium.org/chromium/chromium/src/+/main:net/)
- [11] R. Curtin. (2022) Mlpack: A scalable c++ machine learning library. Peer-reviewed paper on MLpack’s performance. [Online]. Available: <https://jmlr.org/papers/volume23/21-0010/21-0010.pdf>
- [12] PyTorch. (2023) Exporting models to onnx format. PyTorch’s official ONNX export documentation. [Online]. Available: <https://pytorch.org/docs/stable/onnx.html>
- [13] Microsoft. (2023) Onnx runtime performance tuning. Performance benchmarks and optimization guidance. [Online]. Available: <https://onnxruntime.ai/docs/performance/>
- [14] R. Hat. (2024) Podman: A docker alternative. Comparison of Podman and Docker for containerization. [Online]. Available: <https://developers.redhat.com/blog/podman-vs-docker>
- [15] MLCommons. (2024) Mlperf inference benchmarks. Standardized ML performance benchmarking framework. [Online]. Available: <https://mlcommons.org/en/inference-overview/>
- [16] J. Axboe. (2023) Efficient io with io_uring. Technical paper on Linux io_uring for high-speed I/O. [Online]. Available: https://kernel.dk/io_uring.pdf